

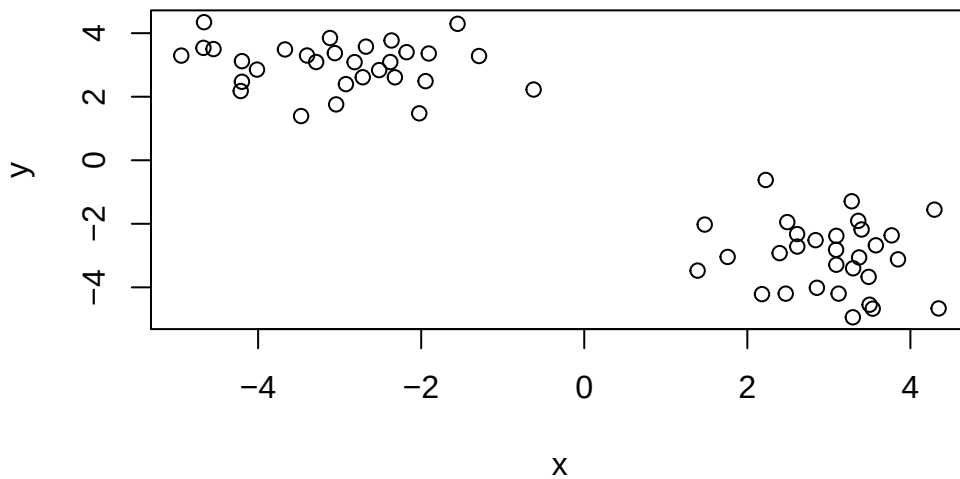
Class07

Rowen (PID: A18363990)

First up kmeans()

Demo of using kmeans() function in base R. First make up some data with a known structure.

```
tmp <- c( rnorm(30, -3), rnorm(30, 3) )  
x <- cbind(x=tmp, y=rev(tmp))  
plot(x)
```



Now we have some made up data in x let's see how kmeans works with this data

```
k <- kmeans(x, centers = 2, nstart = 20)
k
```

K-means clustering with 2 clusters of sizes 30, 30

Cluster means:

	x	y
1	-3.024710	3.002004
2	3.002004	-3.024710

Clustering vector:

[illegible]

Within cluster sum of squares by cluster:

```
[1] 49.937 49.937
(between_SS / total_SS = 91.6 %)
```

Available components:

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"
```

Q. How many points are in each cluster

```
k$size
```

```
[1] 30 30
```

Q. How we do get to the cluster membership/assignment.

```
k$cluster
```

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

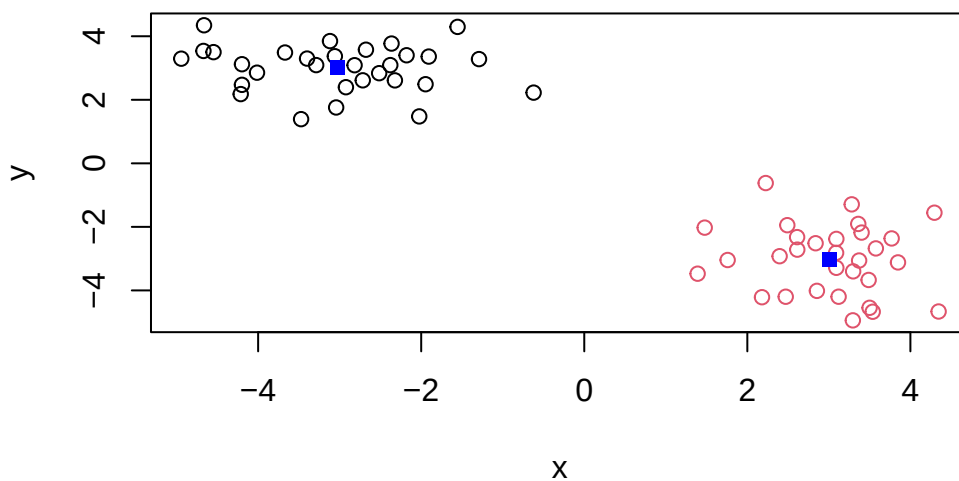
Q. What about cluster centers?

```
k$centers
```

	x	y
1	-3.024710	3.002004
2	3.002004	-3.024710

Now we got to the main results let's use them to plot our data with the kmeans result

```
plot(x, col=k$cluster)
points(k$centers, col="blue", pch=15)
```



Now for Hierarchical Clustering

We will cluster some data `x` with the `hclust()`. In this case `hclust()` requires a distance matrix as input.

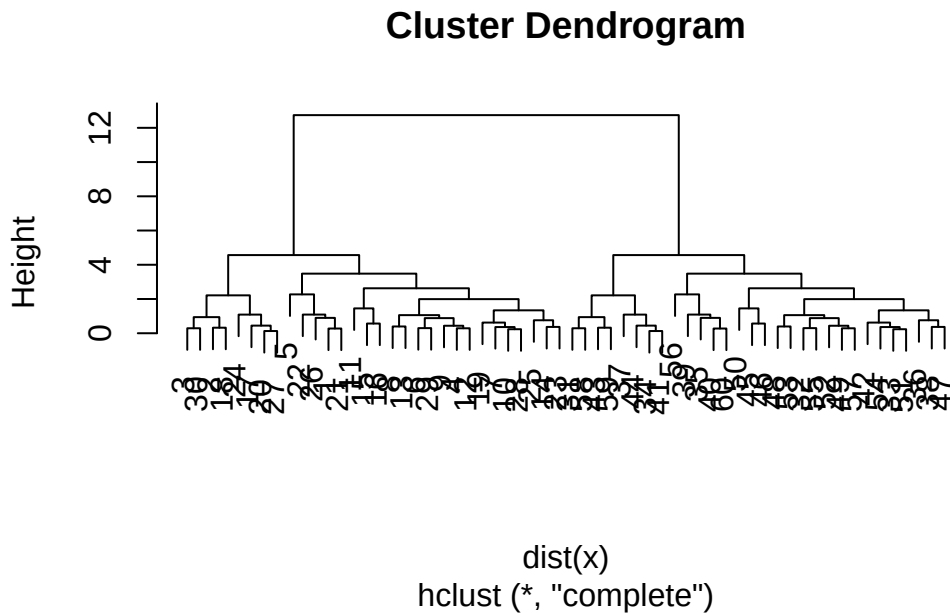
```
hc <- hclust( dist(x) )
hc
```

Call:
`hclust(d = dist(x))`

```
Cluster method   : complete
Distance         : euclidean
Number of objects: 60
```

Let's plot our hclust result

```
plot(hc)
```



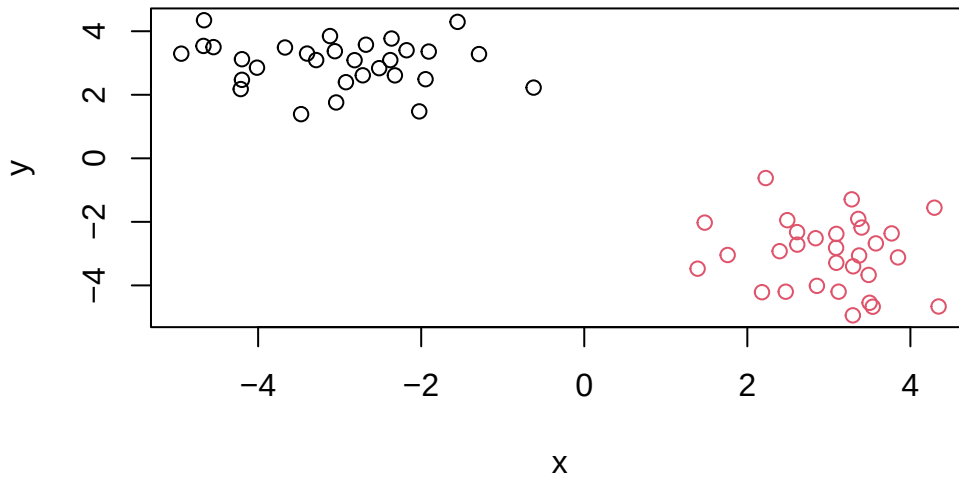
To get our cluster membership vector we need to “cut” the tree with the `cutree()`

```
grps <- cutree(hc, h=8)
grps
```

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

Now plot our data with the `hclust()` results.

```
plot(x, col=grps)
```



Principal Component Analysis (PCA)

PCA of UK food data

Read data from website and try a few visualizations.

```
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url, row.names=1)
x
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139
Fresh_potatoes	720	874	566	1033
Fresh_Veg	253	265	171	143
Other_Veg	488	570	418	355
Processed_potatoes	198	203	220	187

Processed_Veg	360	365	337	334
Fresh_fruit	1102	1137	957	674
Cereals	1472	1582	1462	1494
Beverages	57	73	53	47
Soft_drinks	1374	1256	1572	1506
Alcoholic_drinks	375	475	458	135
Confectionery	54	64	62	41

Q1. How many rows and columns are in your new data frame named x? What R functions could you use to answer this questions?

```
dim(x)
```

```
[1] 17  4
```

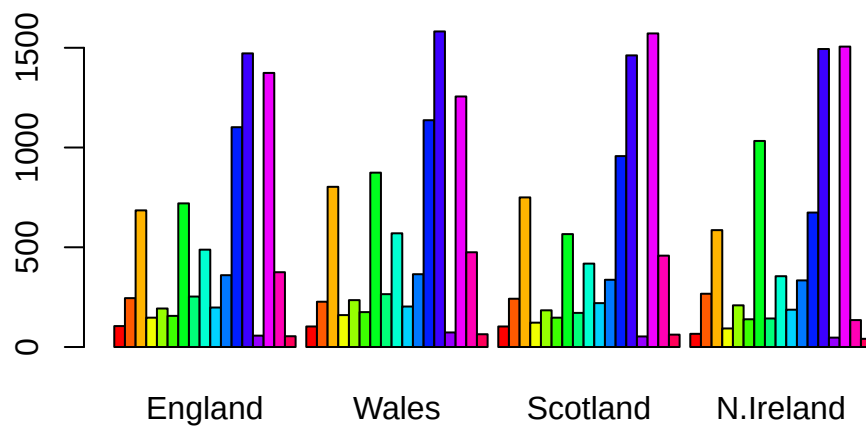
x has 17 rows and 5 columns.

Q2. Which approach to solving the ‘row-names problem’ mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

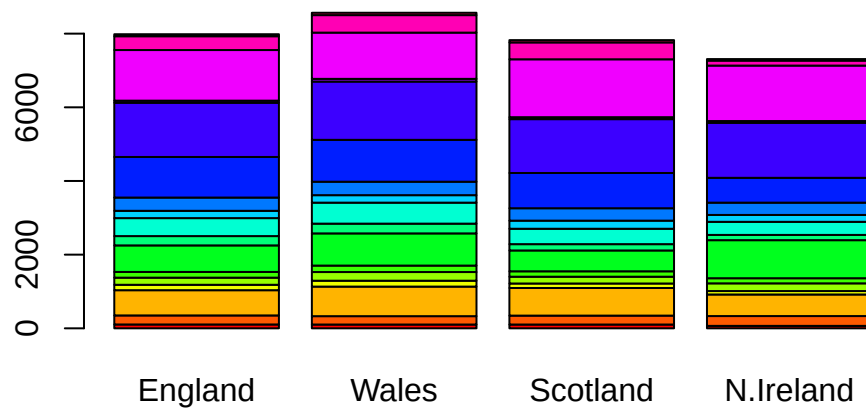
I prefer storing the row names as a column instead of using `x <- x[, -1]`. This is more robust because if you run `x <- x[, -1]` multiple times, it will keep deleting columns and can remove actual data by mistake. Keeping the names as a column is safer and works better for plotting and analysis.

Q3. Changing what optional argument in the above `barplot()` function results in the following plot?

```
barplot(as.matrix(x), beside=T, col=rainbow(nrow(x)))
```



```
barplot(as.matrix(x), beside = FALSE, col = rainbow(nrow(x)))
```



```
library(tidyr)

# Convert data to long format for ggplot with `pivot_longer()`
x_long <- x |>
  tibble::rownames_to_column("Food") |>
  pivot_longer(cols = -Food,
               names_to = "Country",
               values_to = "Consumption")

dim(x_long)
```

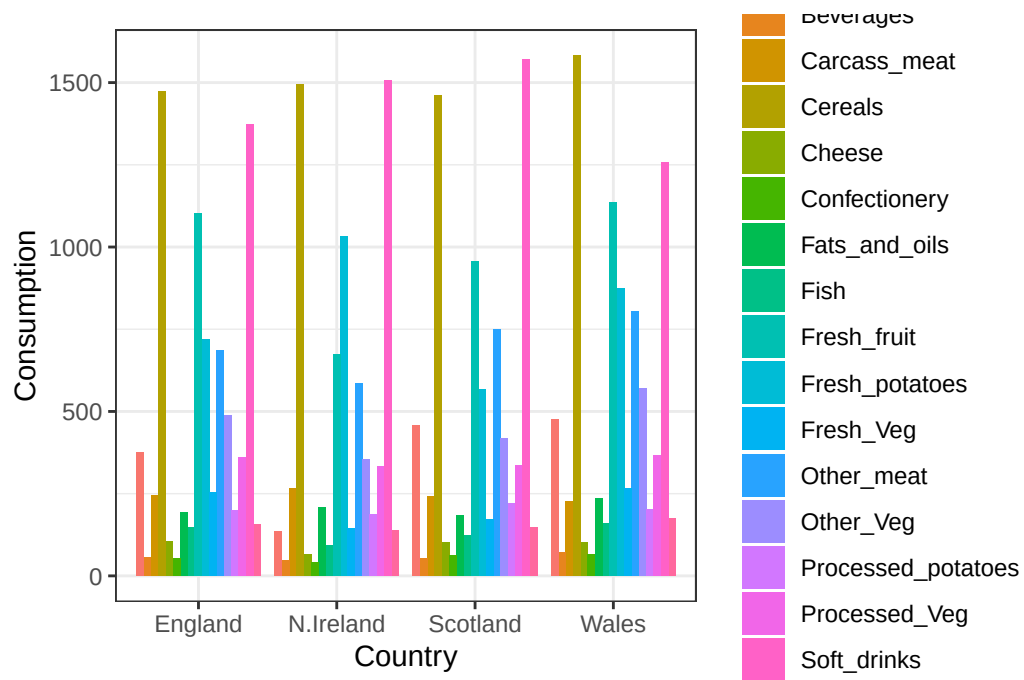
```
[1] 68  3
```

Q4. Changing what optional argument in the above ggplot() code results in a stacked barplot figure?

```
library(ggplot2)
```

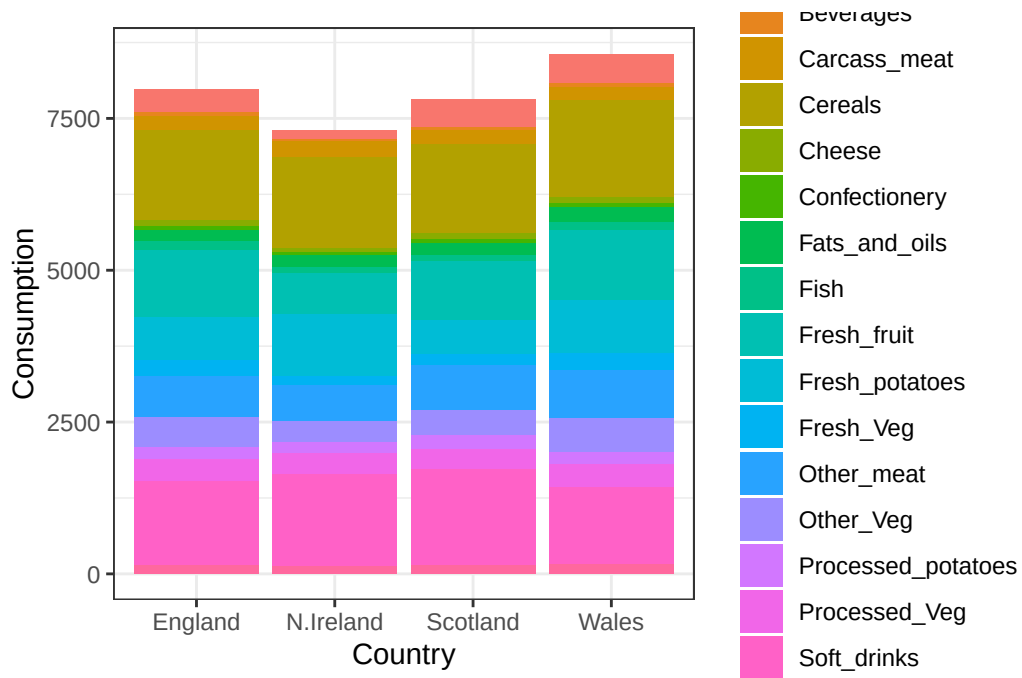
Warning: package 'ggplot2' was built under R version 4.4.3

```
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "dodge") +
  theme_bw()
```



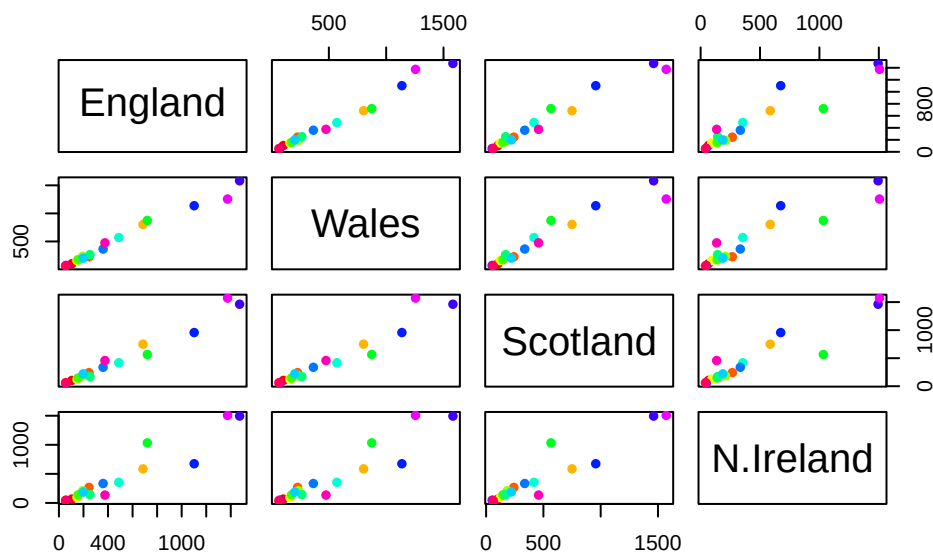
```
library(ggplot2)

ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "stack") +
  theme_bw()
```



Q5. We can use the `pairs()` function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

```
pairs(x, col=rainbow(nrow(x)), pch=16)
```



The `pairs()` function makes scatterplots for every pair of variables, so we can compare countries. Each point is a food category. If a point is on the diagonal, it means the two countries have the same values for that food. Points far from the diagonal mean they are different.

Q6. Based on the `pairs` and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns? Can you easily tell what the main differences between N. Ireland and the other countries of the UK in terms of this data-set?

```
library(pheatmap)
```

England, Scotland, and Wales cluster together, while N. Ireland is more separate. This suggests those three countries have similar food consumption patterns, but N. Ireland is different.

It's not super easy to see exactly which foods are different, but the heatmap shows N. Ireland has different levels for some categories compared to the others.

PCA to the rescue!! The main base R PCA function is called `prcomp()` and we will need to give it the transpose of our input data!

```
pca <- prcomp(t(x))
```

There is a nice summary of how well PCA is doing

```
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	2.921e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.000e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.000e+00

```
attributes(pca)
```

```
$names
```

```
[1] "sdev"      "rotation" "center"    "scale"     "x"
```

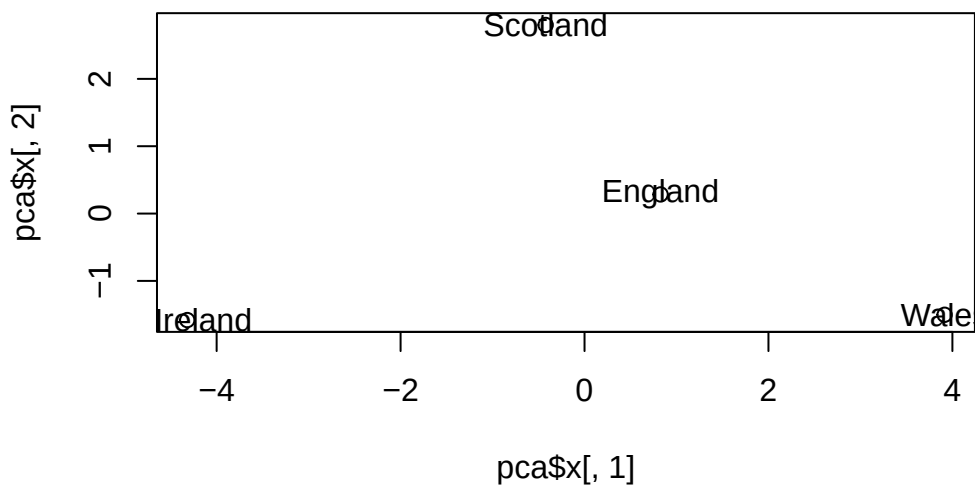
```
$class
```

```
[1] "prcomp"
```

```
pca <- prcomp(t(x), scale. = TRUE)
```

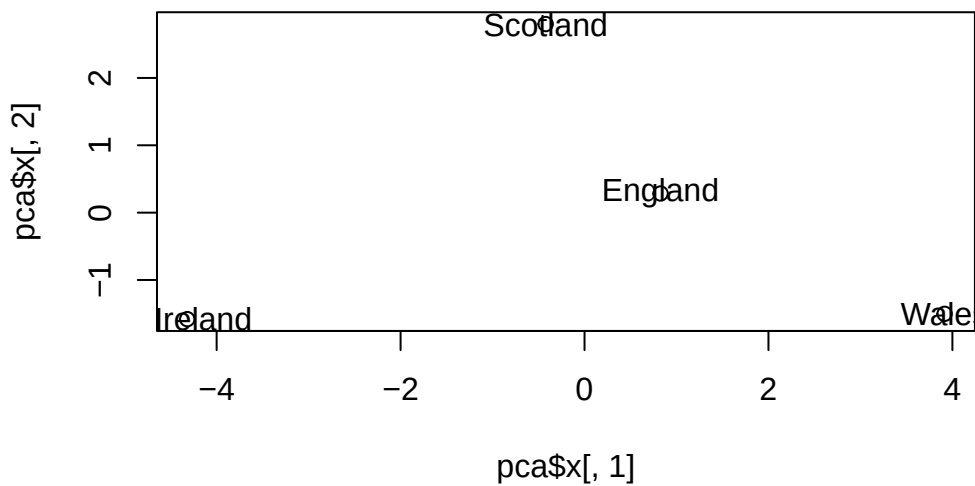
To make our new PCA plot (a.k.a. PCA score plot) we access `pca$x`

```
plot(pca$x[,1], pca$x[,2])  
text(pca$x[,1], pca$x[,2], colnames(x))
```



color up the plot

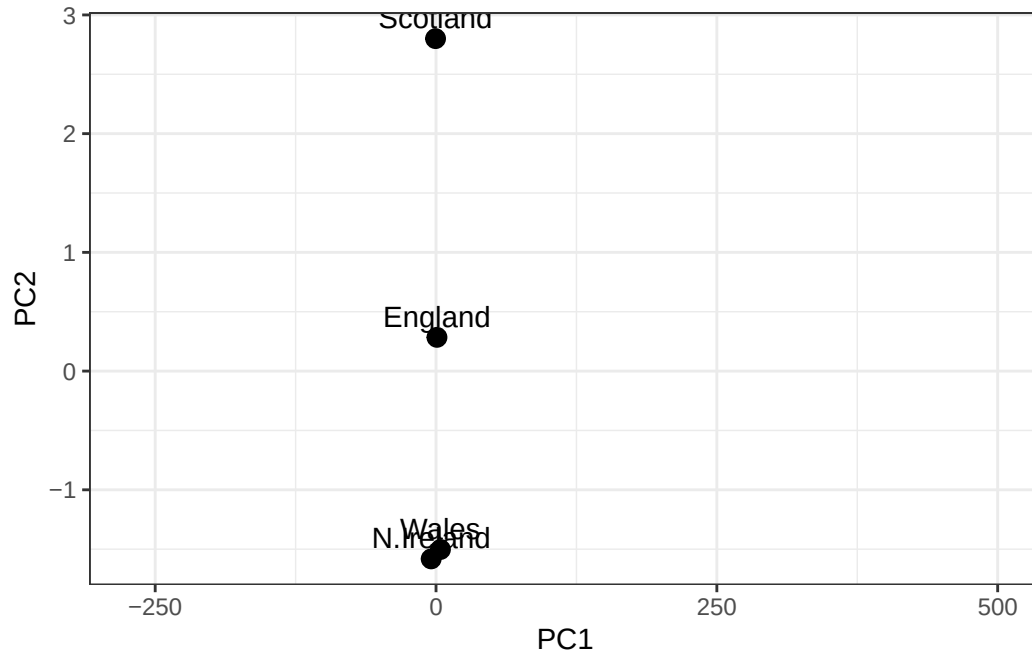
```
country_cols <- c("orange", "red", "blue", "green")
plot(pca$x[,1], pca$x[,2])
text(pca$x[,1], pca$x[,2], colnames(x))
```



Q7. Complete the code below to generate a plot of PC1 vs PC2.
The second line adds text labels over the data points.

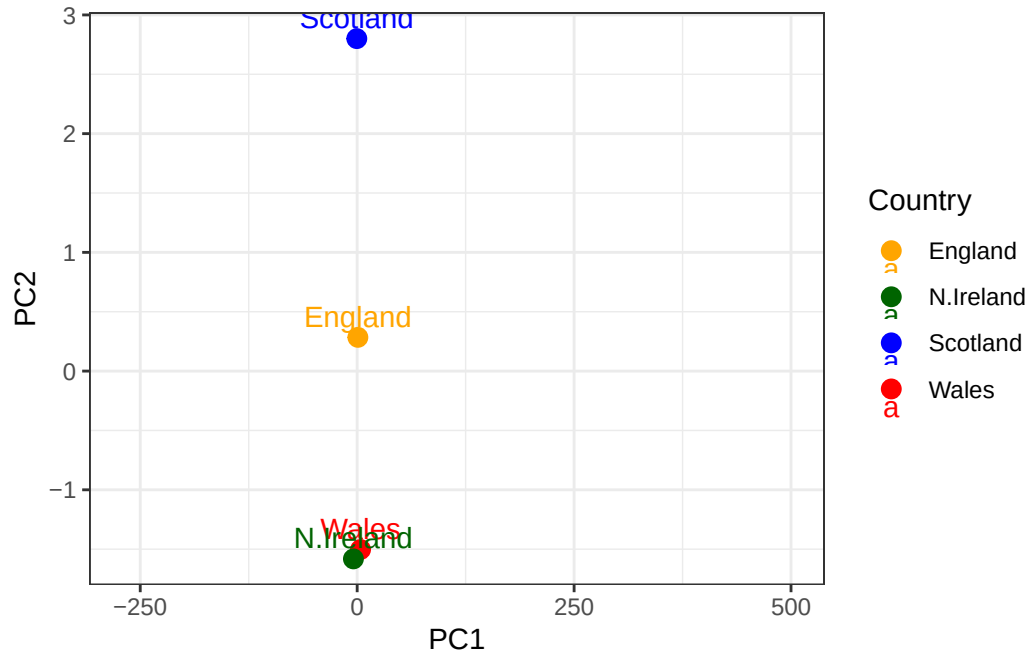
```
df <- as.data.frame(pca$x)
df$Country <- rownames(df)

ggplot(df) +
  aes(x = PC1, y = PC2, label = Country) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



Q8. Customize your plot so that the colors of the country names match the colors in our UK and Ireland map and table at start of this document.

```
ggplot(df) +
  aes(x = PC1, y = PC2, label = Country, color = Country) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  scale_color_manual(values = c(
    "England" = "orange",
    "Scotland" = "blue",
    "Wales" = "red",
    "N.Ireland" = "darkgreen"
  )) +
  theme_bw()
```



```
v <- round( pca$sdev^2/sum(pca$sdev^2) * 100 )
v
```

```
[1] 68 25 7 0
```

```
z <- summary(pca)
z$importance
```

	PC1	PC2	PC3	PC4
Standard deviation	3.408187	2.056239	1.075241	2.448269e-16
Proportion of Variance	0.683280	0.248710	0.068010	0.000000e+00
Cumulative Proportion	0.683280	0.931990	1.000000	1.000000e+00

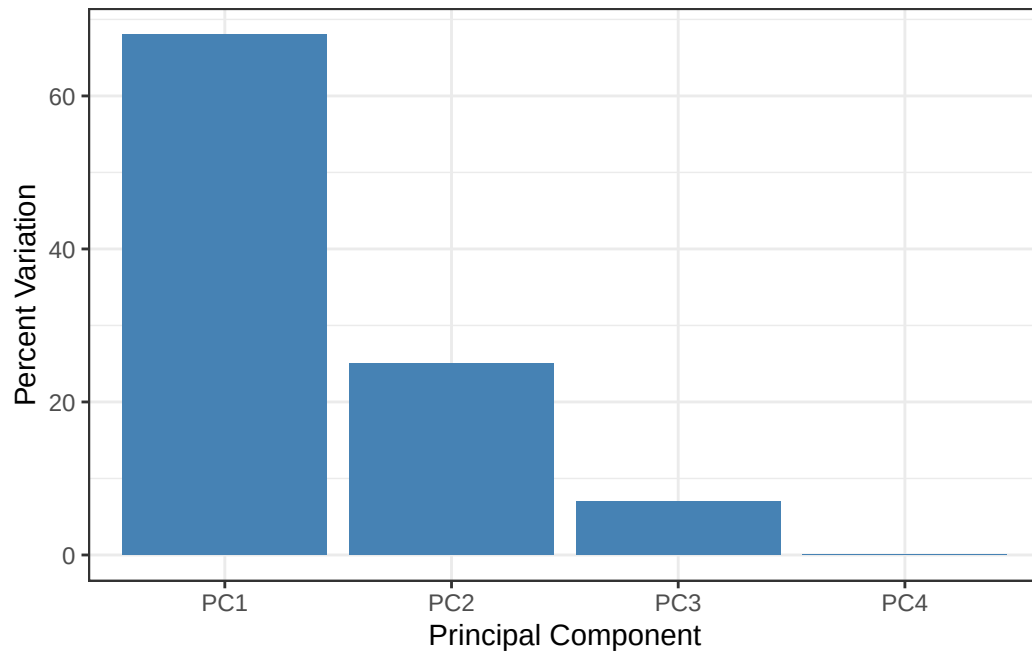
```
variance_df <- data.frame(
  PC = factor(paste0("PC", 1:length(v)), levels = paste0("PC", 1:length(v))),
  Variance = v
)

ggplot(variance_df) +
  aes(x = PC, y = Variance) +
  geom_col(fill = "steelblue") +
```

```

xlab("Principal Component") +
ylab("Percent Variation") +
theme_bw() +
theme(axis.text.x = element_text(angle = 0))

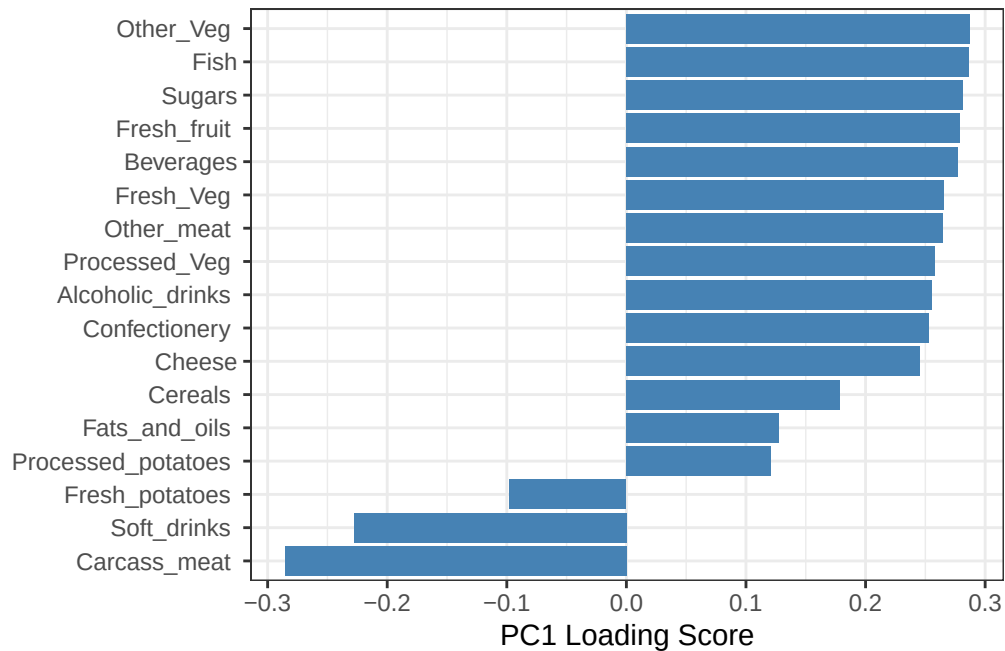
```



```

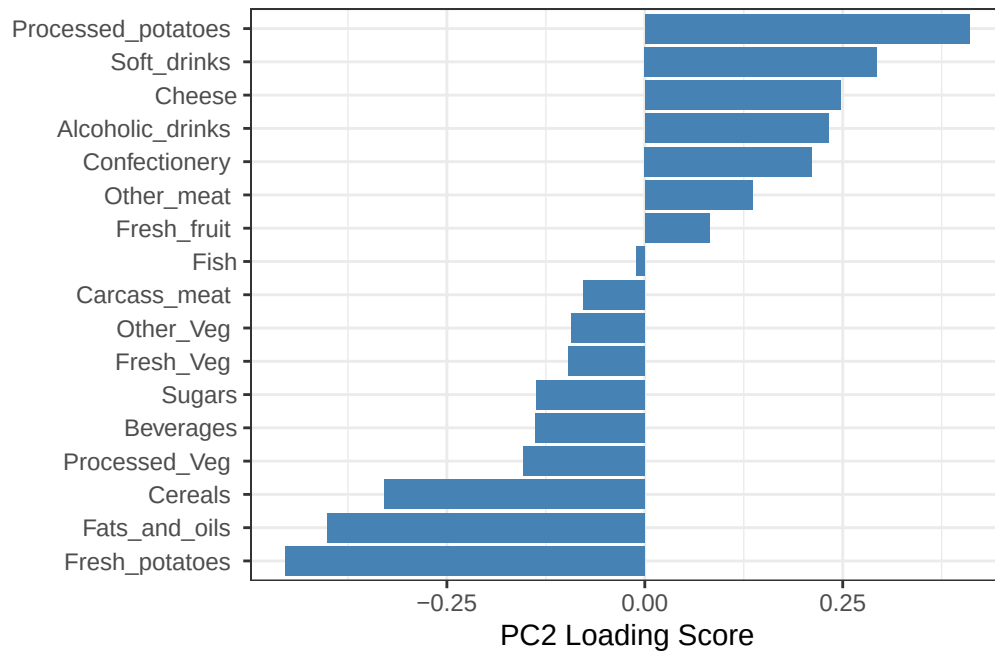
ggplot(pca$rotation) +
  aes(x = PC1,
      y = reorder(rownames(pca$rotation), PC1)) +
  geom_col(fill = "steelblue") +
  xlab("PC1 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))

```



Q9: Generate a similar 'loadings plot' for PC2. What two food groups feature prominently and what does PC2 mainly tell us about?

```
ggplot(pca$rotation) +
  aes(x = PC2,
      y = reorder(rownames(pca$rotation), PC2)) +
  geom_col(fill = "steelblue") +
  xlab("PC2 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```



PC2 is mainly influenced by foods like Fresh_potatoes and Soft_drinks, which have the largest loading values.

This means PC2 separates countries based on differences in these types of foods. Countries with higher values eat more of these foods, while others eat less.

PCA of RNA-Seq data

Read in data from website

```
url2 <- "https://tinyurl.com/expression-CSV"
rna.data <- read.csv(url2, row.names=1)
head(rna.data)
```

	wt1	wt2	wt3	wt4	wt5	ko1	ko2	ko3	ko4	ko5
gene1	439	458	408	429	420	90	88	86	90	93
gene2	219	200	204	210	187	427	423	434	433	426
gene3	1006	989	1030	1017	973	252	237	238	226	210
gene4	783	792	829	856	760	849	856	835	885	894
gene5	181	249	204	244	225	277	305	272	270	279
gene6	460	502	491	491	493	612	594	577	618	638

```
pca <- prcomp( t(rna.data))
summary(pca)
```

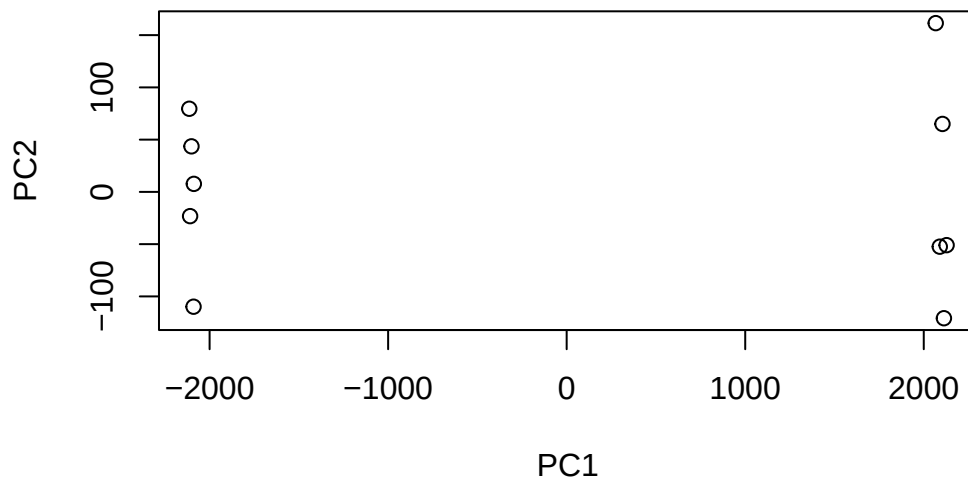
Importance of components:

	PC1	PC2	PC3	PC4	PC5	PC6
Standard deviation	2214.2633	88.9209	84.33908	77.74094	69.66341	67.78516
Proportion of Variance	0.9917	0.0016	0.00144	0.00122	0.00098	0.00093
Cumulative Proportion	0.9917	0.9933	0.99471	0.99593	0.99691	0.99784

	PC7	PC8	PC9	PC10
Standard deviation	65.29428	59.90981	53.20803	2.662e-13
Proportion of Variance	0.00086	0.00073	0.00057	0.000e+00
Cumulative Proportion	0.99870	0.99943	1.00000	1.000e+00

Do our PCA plot of this RNA

```
plot(pca$x[,1], pca$x[,2], xlab="PC1", ylab="PC2")
```



```
plot(pca$x[,1], pca$x[,2], xlab="PC1", ylab="PC2")
text(pca$x[,1], pca$x[,2], colnames(rna.data))
```

